

Teach Module 28: Spring Security Fundamentals

This note was created from our Module 28 teaching chat. The course document was used only to identify the module topic. The teaching content below is independent explanatory material.

Module Focus

Module 28 covers Spring Security fundamentals:

- Authentication
- Authorization
- The security filter chain
- URL-level route security
- Method-level security
- Basic API protection
- JWT overview

Core Ideas

Spring Security is the standard security framework used in Spring Boot applications. Its job is to answer two important questions for every request:

1. Who are you?
2. What are you allowed to do?

Authentication means proving identity.

Authorization means checking permission.

Example:

```
User logs in as alice@example.com
Spring Security verifies the password or token
Spring checks whether Alice can access /admin/users
```

If Alice is logged in but does not have the `ADMIN` role, authentication succeeds but authorization fails.

Why Spring Security Exists

Without a security framework, every controller would need custom checks:

```
if (!user.isLoggedIn()) {
    return "Unauthorized";
}

if (!user.isAdmin()) {
    return "Forbidden";
}
```

That becomes messy and dangerous.

Spring Security centralizes security rules. Instead of scattering checks everywhere, you define rules in one place:

```
/admin/** requires ADMIN
/api/orders/** requires authenticated user
/public/** is open to everyone
```

Spring Security applies those rules before the request reaches the controller.

The Security Filter Chain

Spring Security works mainly through a filter chain.

Before a request reaches a controller, it passes through security filters:

```
HTTP Request
  ↓
Security Filters
  ↓
Authentication check
  ↓
Authorization check
  ↓
Controller
```

For a request such as:

```
GET /api/accounts
```

Spring Security may check:

```
Is there a session or token?
Is the token valid?
Who is the user?
Does this user have permission?
Should this request be blocked?
```

If the request fails security checks, it does not reach the controller.

Authentication

Authentication can happen in different ways:

```
Username/password login
HTTP Basic authentication
Session-based login
JWT token authentication
OAuth2 / OpenID Connect
```

In a simple Spring Boot app, you might start with form login or HTTP Basic authentication.

In modern REST APIs, JWTs are common.

A JWT flow usually looks like this:

```
User sends username/password
Server verifies credentials
Server returns JWT token
Client sends token with future requests
Server validates token on each request
```

Usually the token is sent like this:

```
Authorization: Bearer eyJhbGciOiJIUzI1...
```

Spring Security reads the token, validates it, and builds an authenticated user context.

Authorization

Authorization decides whether an authenticated user can access something.

Spring Security commonly uses roles or authorities.

Example roles:

```
USER
ADMIN
MANAGER
SUPPORT
```

Example users:

```
alice -> USER
bob   -> ADMIN
```

Then you can define rules like:

```
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers("/api/**").authenticated()
.requestMatchers("/public/**").permitAll()
```

Meaning:

```
/admin/** requires ADMIN
/api/**  requires any logged-in user
/public/** is open
```

URL-Level Security

URL-level security protects routes based on request paths.

Example:

```

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults())
        .build();
}

```

This says:

```

/public/** -> anyone can access
/admin/**  -> only ADMIN
everything else -> must be logged in

```

URL-level security is useful for broad access rules.

Method-Level Security

Method-level security protects individual service or controller methods.

Example:

```

@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {
    // delete logic
}

```

Another example:

```

@PreAuthorize("hasAnyRole('USER', 'ADMIN')")
public String viewAccount() {
    return "Account details";
}

```

To enable method-level security:

```

@EnableMethodSecurity

```

Method-level security is useful for business rules and service-layer protection.

401 vs 403

This distinction matters:

```
401 Unauthorized = you are not authenticated
403 Forbidden    = you are authenticated, but not allowed
```

Example:

```
No token provided -> 401
Valid USER token tries /admin -> 403
```

Mental Model

Remember this request flow:

```
Request comes in
Spring Security checks identity
Spring Security checks permission
If allowed, request reaches controller
If not, Spring returns 401 or 403
```

That is the heart of Spring Security.

Practice Exercises

Exercise 1: Secure a Basic Spring Boot App

Create a Spring Boot REST API with these endpoints:

```
GET /public/hello
GET /api/profile
GET /admin/dashboard
```

Security rules:

```
/public/hello -> open to everyone
/api/profile -> any authenticated user
/admin/dashboard -> ADMIN role only
```

Practice goal: understand URL-level authorization.

Exercise 2: Add In-Memory Users

Configure two users:

```
username: user
password: user123
role: USER

username: admin
```

```
password: admin123
role: ADMIN
```

Expected behavior:

```
/public/hello works without login
/api/profile works for user and admin
/admin/dashboard works only for admin
```

Exercise 3: Practice 401 vs 403

Call protected endpoints in different ways:

```
No credentials -> should return 401
USER accessing /admin/dashboard -> should return 403
ADMIN accessing /admin/dashboard -> should return 200
```

Practice goal: clearly understand authentication failure vs authorization failure.

Exercise 4: Create Role-Based Controllers

Create endpoints like:

```
@GetMapping("/user/orders")
public String userOrders() {
    return "User orders";
}

@GetMapping("/manager/reports")
public String managerReports() {
    return "Manager reports";
}

@GetMapping("/admin/users")
public String adminUsers() {
    return "Admin users";
}
```

Then configure:

```
/user/** -> USER, MANAGER, ADMIN
/manager/** -> MANAGER, ADMIN
/admin/** -> ADMIN only
```

Practice goal: design realistic access boundaries.

Exercise 5: Use Method-Level Security

Enable method security and protect service methods:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {
    // delete logic
}
```

Also try:

```
@PreAuthorize("hasAnyRole('USER', 'ADMIN')")
public String viewAccount() {
    return "Account details";
}
```

Practice goal: learn when to protect URLs vs methods.

Exercise 6: Add Password Encoding

Use `BCryptPasswordEncoder` instead of plain text passwords:

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Practice goal: understand why real applications never store raw passwords.

Exercise 7: Build a Login-Based App

Create a small MVC app with:

```
/login
/home
/admin
/logout
```

Configure form login.

Practice goal: understand session-based authentication.

Exercise 8: Build a Stateless API Security Setup

Disable session creation and prepare the app for token-based security:

```
.sessionManagement(session ->
    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
)
```

Practice goal: understand the difference between browser login apps and REST API security.

Exercise 9: JWT Reading Exercise

Create a simple endpoint:

```
GET /api/token-info
```

Send a sample JWT in the `Authorization` header:

```
Authorization: Bearer <token>
```

Your task:

```
Read the Authorization header
Extract the token string
Return whether the token exists
```

You do not need full JWT validation yet.

Practice goal: understand where JWTs appear in requests.

Exercise 10: Custom Access Denied Response

Customize the response for unauthorized or forbidden access.

Return JSON like:

```
{
  "error": "Access denied",
  "status": 403
}
```

Practice goal: make security errors API-friendly.

Lab: Spring Security Basics

Build a small secured Spring Boot REST API.

Goal

Create an API where:

```
Public users can check app status
Logged-in users can view employee data
Admins can create, update, and delete employee data
```

Requirements

Create these endpoints:

```
GET    /public/status
GET    /employees
GET    /employees/{id}
POST   /employees
```



```
PUT      /employees/{id}
DELETE  /employees/{id}
```

Access rules:

```
/public/status      -> no login required
GET /employees      -> USER or ADMIN
GET /employees/{id} -> USER or ADMIN
POST /employees     -> ADMIN only
PUT /employees/{id} -> ADMIN only
DELETE /employees/{id} -> ADMIN only
```

Create two users:

```
username: user
password: user123
role: USER

username: admin
password: admin123
role: ADMIN
```

Step 1: Create Project

Use Spring Initializr or your IDE.

Dependencies:

```
Spring Web
Spring Security
Spring Boot DevTools
```

Optional:

```
Lombok
```

Step 2: Create Employee Model

```
public class Employee {
    private Long id;
    private String name;
    private String department;

    public Employee(Long id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }
}
```

```

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDepartment() {
        return department;
    }
}

```

Step 3: Create Controller

```

@RestController
public class EmployeeController {

    private final List<Employee> employees = new ArrayList<>();

    public EmployeeController() {
        employees.add(new Employee(1L, "Alice", "Engineering"));
        employees.add(new Employee(2L, "Bob", "Finance"));
    }

    @GetMapping("/public/status")
    public String status() {
        return "Application is running";
    }

    @GetMapping("/employees")
    public List<Employee> getEmployees() {
        return employees;
    }

    @GetMapping("/employees/{id}")
    public Employee getEmployeeById(@PathVariable Long id) {
        return employees.stream()
            .filter(employee -> employee.getId().equals(id))
            .findFirst()
            .orElseThrow();
    }

    @PostMapping("/employees")
    public String createEmployee(@RequestBody Employee employee) {
        employees.add(employee);
        return "Employee created";
    }

    @PutMapping("/employees/{id}")
    public String updateEmployee(@PathVariable Long id, @RequestBody Employee

```

```

updatedEmployee) {
    return "Employee updated";
}

@DeleteMapping("/employees/{id}")
public String deleteEmployee(@PathVariable Long id) {
    employees.removeIf(employee -> employee.getId().equals(id));
    return "Employee deleted";
}
}

```

Step 4: Create Security Configuration

```

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/employees/**").hasAnyRole("USER",
"ADMIN")
                .requestMatchers(HttpMethod.POST, "/employees/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PUT, "/employees/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.DELETE,
"/employees/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults())
            .build();
    }

    @Bean
    UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
        UserDetails user = User.builder()
            .username("user")
            .password(passwordEncoder.encode("user123"))
            .roles("USER")
            .build();

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder.encode("admin123"))
            .roles("ADMIN")
            .build();

        return new InMemoryUserDetailsManager(user, admin);
    }
}

```

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}
```

Required imports include:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
```

Step 5: Test the API

Public endpoint:

```
curl http://localhost:8080/public/status
```

Expected:

```
Application is running
```

Try employees without login:

```
curl http://localhost:8080/employees
```

Expected:

```
401 Unauthorized
```

Try employees as normal user:

```
curl -u user:user123 http://localhost:8080/employees
```

Expected:

Employee list returned

Try creating employee as normal user:

```
curl -u user:user123 \  
-X POST http://localhost:8080/employees \  
-H "Content-Type: application/json" \  
-d '{"id":3,"name":"Carol","department":"HR"}'
```

Expected:

403 Forbidden

Try creating employee as admin:

```
curl -u admin:admin123 \  
-X POST http://localhost:8080/employees \  
-H "Content-Type: application/json" \  
-d '{"id":3,"name":"Carol","department":"HR"}'
```

Expected:

Employee created

Deliverables

Submit:

```
Employee.java  
EmployeeController.java  
SecurityConfig.java  
Screenshots or terminal output proving:  
- /public/status works without login  
- /employees fails without login  
- /employees works as USER  
- POST /employees fails as USER  
- POST /employees works as ADMIN
```

Challenge

Add this rule:

Only ADMIN can access /admin/**

Create:

```
@GetMapping("/admin/report")  
public String adminReport() {  
    return "Admin report";  
}
```

Then test it with both `user` and `admin` .